

Guía del repositorio

Laboratorio Estructural Digital — Grupo 7

Edificio de Ingeniería UAndes · OpenSees + Unity

Qué es cada archivo · qué hace cada código

El flujo completo, en el orden en que corre

Cómo activar y ejecutar cada cosa

Métodos Computacionales en Obras Civiles · 2026-02

Todos los comandos de esta guía fueron ejecutados antes de escribirla.

1. Qué es esto, en una página

Tenemos los **planos reales** del Edificio de Ingeniería de la UAndes, en archivos de AutoCAD. El proyecto los lee, arma con ellos un **modelo estructural**, lo resuelve con un motor de cálculo llamado **OpenSees**, comprueba que el resultado tenga sentido, y lo muestra en **3D en Unity**.

El edificio son en realidad **dos cuerpos**, construidos en épocas distintas y separados por una *junta de dilatación* (una ranura de 5 cm que los deja moverse independientes en un sismo):

Cuerpo	Planos	Carpeta	Tamaño del modelo
lt2 — el nuevo	2024_22	edificios/lt2/	232 nodos, 378 elementos
ingenieria — el antiguo	2017_67	edificios/ingenieria/	326 nodos, 559 elementos
conjunto — los dos	calce.json	edificios/conjunto/	558 nodos, 937 elementos

La regla de oro del proyecto. Hay una separación estricta que no se rompe nunca:

OpenSees (Python)

→

archivos JSON

→

Unity

CALCULA

la fuente de verdad

solo MUESTRA

Unity **nunca** calcula estructura. Si hay que calcular algo, se calcula en Python y viaja por JSON. Esto es lo primero que puede preguntar el profesor.

Las dos palabras que hay que entender

edificio En todos los comandos hay que decir sobre cuál de los tres se trabaja: `lt2` , `ingenieria` o `conjunto` .

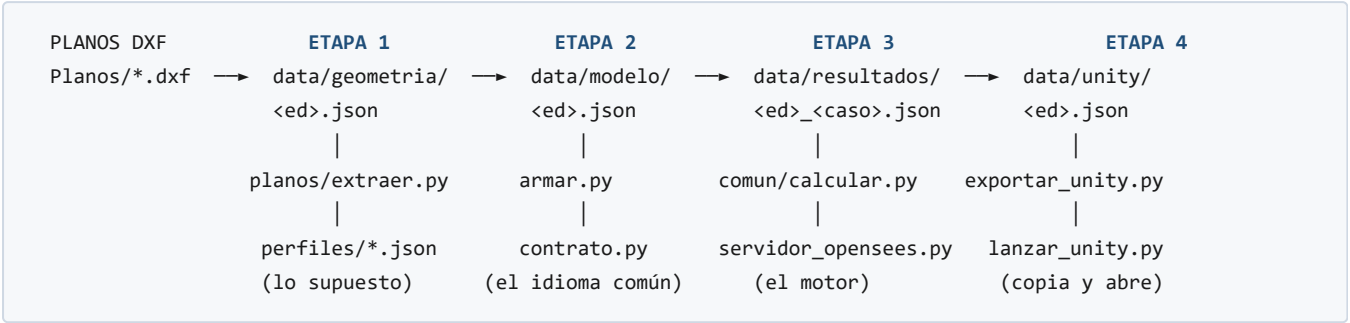
caso Es una situación de carga. Hay cuatro: **G** = el peso propio del edificio más la losa; **Q** = la gente y los muebles (sobrecarga de uso); **EX** y **EY** = el sismo empujando en cada dirección.

Por qué el proyecto está partido en cuatro etapas

Podría ser un solo programa gigante que lea los planos y muestre el edificio. No lo es, y a propósito: cada etapa deja su resultado en un archivo **JSON** que se puede abrir, mirar y verificar por separado. Si algo sale mal, se sabe *en qué etapa*.

Además, la etapa del medio — `data/modelo/` — es donde los dos edificios dejan de ser "planos 2024_22" y "planos 2017_67" y pasan a ser simplemente listas de nodos y barras. **Ahí hablan el mismo idioma**, y por eso unirlos es un script corto en vez de fusionar dos programas.

2. El flujo completo, en orden



Cada etapa lee lo que dejó la anterior. A continuación, qué pasa exactamente en cada una.

1 Leer los planos → data/geometria/lt2.json

```
python edificios\planos\extraer.py "..\Planos\LT2_CAL_dxf" --perfil lt2_2024_22 (ver 4.7)
```

Qué hace: abre los 22 archivos DXF de AutoCAD y saca de ahí la geometría: dónde están los ejes, a qué altura está cada piso, dónde hay muros, pilares y vigas. El resultado es "lo que dice el plano", todavía sin decisiones de modelado.

El archivo que manda: edificios/lt2/planos/extraer.py , que llama a los demás en este orden:

Llama a	Y ese archivo hace...
perfil.py	Lee perfiles/lt2_2024_22.json : qué capas del DXF son muros, cuáles vigas, qué zona del dibujo interesa. Nada de esto está escrito en el código , así que sirve para otro juego de planos.
lectura.py	Abre una lámina y la deja en metros. Los DXF están en centímetros y casi todo el dibujo vive dentro de <i>bloques</i> y <i>referencias externas</i> , que hay que "explotar".
ejes.py	Saca los ejes con su nombre (A, B, 1, 2...). Trampa: el globo con la letra no siempre está sobre su eje; cuando dos ejes quedan muy juntos, el dibujante corre el globo y lo une con un quiebre.
alineacion.py	Cada lámina está dibujada en su propio origen. Este archivo las registra entre sí usando los ejes que comparten. Sin esto, la fundación queda corrida 5 m respecto de los pilares, en silencio.
muros.py	Los muros, que en el plano son dos líneas paralelas (sus caras) o una zona achurada. También une los muros que el dibujo parte en dos por un cruce.
pilares.py	Los pilares: rectángulos cerrados, con su rótulo (P.70x70).
vigas.py · losas.py	Las vigas y el espesor de las losas, desde sus rótulos.
niveles.py	Las cotas de cada piso, desde las elevaciones. Verifica que el desfase del dibujo sea el mismo para todas.

enfierradura.py

El **fierro**: estribos y trabas de los 40 pilares, malla y barras de borde de 29 muros. Lo lee de las elevaciones, donde está escrito bajo el rótulo de la sección.

Deja: `data/geometria/lt2.json` y un informe de auditoría en `reports/` con todo lo que quedó dudoso.

Esta etapa casi nunca se corre. Solo si cambian los planos o el perfil. Es la más lenta y su resultado ya está guardado en el repositorio.

2 Armar el modelo → `data/modelo/lt2.json`

```
python edificios\lt2\armar.py
```

Qué hace: convierte "lo que dice el plano" en un **modelo estructural**: nodos, barras, secciones, apoyos y cargas. Acá se toman las decisiones de modelado.

La cadena de llamadas:

<pre>armar.py ├─ exportar_unity.construir() │ └─ modelo_lt2.ModeloLT2() │ ├── .preparar() │ │ ├── malla.py │ │ └─ panos.py │ ├── .ensamblar('G') │ └─ .resolver() ├─ contrato.separar() └─ contrato.validar()</pre>	arma el modelo y lo resuelve una vez geometría → nodos y barras corta las vigas donde de verdad se cruzan reparte la losa a 45° secciones, diafragmas, cargas lo pasa a OpenSees parte en "estructura" y "dibujo" busca errores que no dan error
---	--

Archivo	Qué resuelve
malla.py	El dibujante corta las polilíneas donde se le ocurre; el modelo necesita que las vigas se corten en sus intersecciones reales . También engancha cada muro al marco con <i>brazos rígidos</i> .
panos.py	Los "paños" de losa no vienen dados en el plano: se encuentran como las caras del grafo que forman las vigas. Después reparte el peso de la losa a las vigas trazando bisectrices a 45°.
modelo_lt2.py	El corazón. Crea las secciones, los <i>diafragmas rígidos</i> (que hacen que cada piso se mueva como una placa), y aplica las tres vías de carga: peso propio, losa repartida, y el peso de los muros como carga puntual.
contrato.py	Define qué es un "modelo" para este proyecto. <code>separar()</code> aparta lo que es estructura de lo que es solo dibujo. <code>validar()</code> caza el error clásico: una carga apuntando a una barra que ya no existe — OpenSees la descarta sin avisar y el edificio pesa menos de lo que debería.

Lo que imprime:

```
caso G resuelto. Carga total 34148.98 kN
enfierradura: 40 columnas, 29 de 40 muros
232 nodos, 378 elementos, 13 secciones, 5 diafragmas, 4 caso(s)
243 elementos reciben losa, 2515.89 m2 en total
-> data\modelo\lt2.json
```

3 Resolver → data/resultados/lt2_G.json y tres más

```
python comun\calcular.py lt2
```

Qué hace: resuelve los cuatro casos de carga y guarda, para cada uno, cuánto se mueve cada nodo (*desplazamientos*), cuánta fuerza aguanta cada apoyo (*reacciones*) y qué esfuerzo tiene cada barra (*fuerzas internas*).

Archivo	Qué hace
calcular.py	Orquesta. No sabe de qué edificio se trata: lee data/modelo/<ed>.json y funciona con cualquiera. Después de resolver, comprueba el equilibrio.
servidor_opensees.py	El motor. Construye el modelo dentro de OpenSees y lo resuelve. También puede correr como servidor , para recalculer en vivo desde Unity.

La trampa más peligrosa del proyecto está acá. Para comprobar el equilibrio uno suma todas las reacciones y las compara con la carga aplicada. Pero un nodo que pertenece a un *diafragma* devuelve también la fuerza de esa restricción, que es **interna**. Sumando todo, el corte basal del LT2 da **7266 kN** cuando el real es **3633 kN**: el doble exacto. La separación correcta es **por grado de libertad**, y está en `calcular.equilibrio()`.

Lo que imprime:

```
equilibrio Fz: aplicada -34148.979 reaccion 34148.979
peor error 2.0e-04 kN sobre 34149.0 kN (5.9e-09 relativo)
-> data/resultados\lt2_G.json
```

4 Exportar a Unity → data/unity/lt2.json

```
python edificios\lt2\exportar_unity.py
```

Qué hace: junta el modelo con sus resultados y lo deja en la forma exacta que espera el visor: cada nodo con su desplazamiento adentro, cada barra con sus *ejes locales* ya calculados, y los polígonos de área tributaria para poder ver de dónde sale la carga de cada viga.

Unity no deduce nada. La orientación de cada sección se calcula en Python y viaja en el JSON. Si Unity tuviera que adivinarla, tarde o temprano dibujaría algo distinto de lo que se calculó.

5 Ver el edificio

```
python comun\lanzar_unity.py app conjunto --pantalla-completa
```

lanzar_unity.py copia el JSON a la carpeta que Unity lee (StreamingAssets/) **con el nombre que la escena declara**, y abre la aplicación.

Por qué eso del nombre. El visor tiene el nombre del archivo escrito en la escena de Unity, no en el código. Durante días el lanzador copió a un nombre y el visor abrió otro: la app arrancaba, mostraba un edificio viejo, y no daba ningún error. Ahora el lanzador **lee de la escena** qué archivo hay que escribir.

+ Unir los dos edificios

```
python edificios\conjunto\armar.py      después de armar los dos por separado
```

Lee los dos data/modelo/ , los pone en el mismo sistema de coordenadas según calce.json , y renumera para que no choquen los identificadores.

Concepto	Qué significa
El calce	Cada juego de planos está dibujado en su propio marco. calce.json declara el corrimiento (dx , dy , dz) y cómo se midió cada uno : el dy sobre tres ejes que ambos planos comparten, el dx derivado de las caras más la junta declarada.
Junta libre	Ningún elemento cruza la junta. Los dos cuerpos se resuelven independientes aunque estén en el mismo archivo.
E por cuerpo	El LT2 es hormigón G35 y el otro G28. Como el formato tiene un solo material, al unirlos el LT2 corría con 28 MPa: 10.6 % más blando . Se arregló poniéndole a cada sección su propio módulo elástico.

La verificación que lo atrapó. Si la junta es libre, cada cuerpo dentro del conjunto tiene que dar **exactamente** lo mismo que resuelto solo. Eso es lo que comprueba verificar_conjunto.py , y es lo que el equilibrio no puede ver: la carga que baja al suelo era la misma, solo cambiaba cuánto se deformaba el edificio para bajarla.

3. Catálogo de archivos

comun/ — sirve para cualquier edificio

Archivo	Qué hace
<code>rutas.py</code>	El único que sabe dónde está cada carpeta. Encuentra la raíz del proyecto subiendo hasta encontrar la marca del repositorio, en vez de contar carpetas hacia atrás (que falla en silencio si un archivo se mueve).
<code>contrato.py</code>	Define el formato neutro del modelo. Separa estructura de dibujo, valida, sella el área tributaria y los ejes locales dentro de cada elemento, y normaliza los polígonos al formato que lee Unity.
<code>servidor_opensees.py</code>	El motor de cálculo, y el servidor Flask para recalcular en vivo desde Unity.
<code>calcular.py</code>	Etapa 3. Resuelve los cuatro casos y verifica el equilibrio separando por grado de libertad.
<code>combinar.py</code>	Superposición. Combina los casos ya resueltos ($R = \sum \lambda R$) y comprueba el resultado contra una corrida real de OpenSees con la carga combinada, sobre <i>todos</i> los grados de libertad.
<code>sismo.py</code>	Revisa un caso de sismo: carga aplicada, corte basal, hacia dónde se deforma el edificio, cuánto gira cada piso (<i>torsión</i>) y dónde está el centro de rigidez.
<code>capacidad.py</code>	Cuánto aguanta una sección. Arma la sección de fibras desde el modelo y calcula la curva momento-curvatura y el diagrama de interacción P-M. Sirve para columnas y para muros.
<code>verificar_tributarias.py</code>	Comprueba que la losa que se aplica sea la misma que se dibuja, mirando que la presión implícita sea constante dentro de cada piso.
<code>test_contrato_unity.py</code>	Comprueba que cada dato del JSON tenga su campo en el código C#. Unity ignora en silencio lo que no reconoce, y un campo mal escrito da una deformada plana sin ningún error.
<code>verificar_todo.py</code>	Corre las 34 comprobaciones del proyecto y resume. Es el comando de antes de entregar.
<code>lanzar_unity.py</code>	Regenera, copia y abre el visor.

edificios/lt2/ — el cuerpo nuevo

Archivo	Qué hace
<code>perfiles/lt2_2024_22.json</code>	Todo lo específico y todo lo supuesto , con su razón escrita: capas del DXF, zona de interés, cargas del plano, parámetros de sismo, dinteles, diámetro del fierro longitudinal.
<code>planos/*.py</code>	El ingestor de DXF. Ver la etapa 1.
<code>mall.py</code> · <code>panos.py</code>	La malla de vigas y el reparto de la losa. Ver la etapa 2.

modelo_lt2.py	Arma el modelo en OpenSees.
armar.py · exportar_unity.py	Etapas 2 y 4.
verificar_lt2.py	59 comprobaciones del modelo: secciones contra cálculo a mano, equilibrio, orientación de muros, linealidad, diafragmas, brazos rígidos, zonas sin apoyo, área de losa piso contra piso, los cuatro casos y las derivas contra NCh433.
tests/	La lectura del DXF y el reanálisis con el servidor.

edificios/ingenieria/ — el cuerpo antiguo

Archivo	Qué hace
planos_v2.py	Lee sus DXF: ejes con quiebre de globo, muros dibujados como línea y como achurado, registro entre láminas.
benchmark_3d.py	El modelo: la grilla de pórticos, el subterráneo (que es una <i>zona</i> , no media planta), la fundación escalonada y los dos voladizos metálicos.
enfierradura.py	La armadura de sus 82 columnas, trazable a la lámina de detalles típicos.
armar.py · export_unity.py	Etapas 2 y 4.
verificar_planos.py	Compara el modelo contra los DXF y falla si algo se aparta más de 1 cm.

edificios/conjunto/ · semana03/

Archivo	Qué hace
conjunto/calce.json	La transformación entre los dos planos, con cómo se midió cada número.
conjunto/armar.py	Une los dos modelos y mide que la junta sea la declarada y las cotas coincidan.
conjunto/verificar_conjunto.py	Cada cuerpo dentro del conjunto = el cuerpo solo, exacto.
semana03/parametros.json	Los valores que define el profesor: carga viva (con la Tabla 4 de NCh1537 adentro), coeficiente sísmico, patrón de reparto en altura, combinaciones. Cada número con la fuente de dónde sale.
semana03/lab_semana03.py	Las Partes A, B y C del laboratorio, sobre cualquier edificio. Arma Q, EX y EY en memoria con los parámetros pedidos y le pide las verificaciones a <code>sismo.py</code> y <code>combinar.py</code> .
semana03/verificar_rc.py	Compara la sección de fibras contra el cálculo a mano del curso de hormigón armado.
semana03/demanda_capacidad.py	Pone el punto (P, M) de cualquier columna o muro sobre su curva de capacidad. Arma la demanda con los parámetros de la Semana 3, no con un resultado guardado. Es el gráfico de la defensa.
semana03/COMANDOS.md	La chuleta de la interrogación: la secuencia de comandos en orden, los cuatro números y las respuestas que van a pedir.
semana03/verificar_viga_partida.py	Demuestra que una viga que se ve hundida en el visor no está mal modelada.

4. Cómo correr cada cosa

4.1 Preparar el computador — una sola vez

Abrir **PowerShell** en la carpeta del proyecto y correr:

```
cd "C:\...\P1\A1P3_Grupo_7"
.\setup.ps1
```

Eso crea una carpeta `.venv` (el "entorno virtual": una instalación de Python privada del proyecto) e instala lo que hace falta: OpenSeesPy, ezdxf, matplotlib y Flask. Tarda unos minutos.

Si PowerShell responde *"la ejecución de scripts está deshabilitada"*, correr antes, en esa misma ventana:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

4.2 Activar el entorno — cada vez que se abre una terminal

Hay dos formas. **La segunda es la recomendada**, porque no hay que acordarse de nada.

```
# FORMA A - activarlo
.\.venv\Scripts\Activate.ps1    # el prompt queda con (.venv) adelante
python comun\verificar_todo.py  # y ya se escribe "python" a secas
deactivate                      # para salir

# FORMA B - no activar nada, llamar al Python del proyecto
.\.venv\Scripts\python.exe comun\verificar_todo.py
```

En el resto de esta guía se escribe `python` por brevedad. Si no activaste el entorno, poné `.\.venv\Scripts\python.exe` en su lugar.

4.3 ¿Está todo bien? — el comando de siempre

```
python comun\verificar_todo.py      # las 34 comprobaciones, ~4 minutos
python comun\verificar_todo.py --rapido # sin las lentas
python comun\verificar_todo.py --solo sismo combinar
```

Si termina diciendo `34 de 34 EN OK`, el repositorio está como se entrega. **Correlo antes de subir cualquier cambio**. Si algo falla, muestra la salida del script que falló, sin esconder nada.

4.4 Ver el edificio en 3D

```
python comun\lanzar_unity.py app conjunto --pantalla-completa
python comun\lanzar_unity.py app lt2
python comun\lanzar_unity.py app ingenieria
```

La primera vez compila la aplicación (varios minutos). Después arranca en segundos.

Modo	Para qué
app <edificio>	Abrir el visor. Es el de la demostración.
editor	Abrir el proyecto en Unity para trabajar en el visor. Hay que apretar Play a mano.
build --forzar	Recompilar la app después de cambiar un archivo <code>.cs</code> .
servidor	Levantar el servidor de recálculo.

Atajo para el LT2, que exporta y abre de una:

```

.\ver.ps1           # exporta el LT2 y abre el visor
.\ver.ps1 -Servidor  # además levanta el servidor de recálculo
.\ver.ps1 -Recompilar # recompila la app antes de abrir
.\ver.ps1 -SoloExportar # solo regenera el JSON

```

Para **editar en Unity y recalcular en vivo**, en otra ventana de PowerShell:

```

python comun\servidor_opensees.py      # solo este computador
python comun\servidor_opensees.py --lan # abierto a la red local (celular, AR)

```

4.5 Los diagramas de capacidad

Todos los gráficos salen a la carpeta `semana03/resultados/` como archivos PNG.

Paso 1 — ver qué elementos se pueden analizar

```
python semana03\demanda_capacidad.py lt2 --lista
python semana03\demanda_capacidad.py ingenieria --lista
```

Lista cada columna y muro con armadura, con su posición y cuántas barras tiene.

Paso 2 — el diagrama P-M con el punto de demanda encima

```
python semana03\demanda_capacidad.py lt2 1 --grafico          # una columna
python semana03\demanda_capacidad.py lt2 9 --grafico          # un muro
python semana03\demanda_capacidad.py ingenieria 18 --grafico
```

Deja `pm_<edificio>_<elemento>.png`: la curva de lo que la sección aguanta, y encima un punto por cada caso de carga. Si el punto está dentro de la curva, la sección resiste.

Paso 3 — con lo que dicte el profesor

```
python semana03\demanda_capacidad.py lt2 1 --comb 1.2 1.6 1.0 0.3 --grafico
python semana03\demanda_capacidad.py ingenieria 18 --uso oficinas --grafico
```

Los cuatro números de `--comb` son los factores de G, Q, EX y EY, en ese orden. Este script **arma Q, EX y EY con los mismos parámetros del laboratorio** y los resuelve en la corrida, así que también acepta `--uso`, `--q`, `--cs` y `--patron`: el punto que se dibuja es siempre el del edificio que se acaba de verificar.

Todos los elementos de una vez

```
python semana03\demanda_capacidad.py lt2 --todas --comb 1.2 1.6 1.0 0.3
python semana03\demanda_capacidad.py ingenieria --todas
```

Una tabla con la *utilización* de cada elemento: cuánto de su capacidad está usando. Si es menor que 1, pasa.

La sección por dentro: fibras, M-φ y P-M

```
python comun\capacidad.py lt2 1          # resumen y momento-curvatura
python comun\capacidad.py lt2 1 --pm      # + la curva P-M, interpretada
python comun\capacidad.py lt2 1 --mphi    # + M-φ a varias compresiones
python comun\capacidad.py lt2 1 --dibujo   # + cómo se cortó en fibras
python comun\capacidad.py lt2 1 --sensibilidad # ¿alcanzan 20 fibras?
python comun\capacidad.py ingenieria 18 --pm --mphi --dibujo
```

Archivo que deja	Qué muestra
<code>fibras_<ed>_<n>.png</code>	La sección cortada en fibras, con el núcleo confinado, el recubrimiento y cada barra en su sitio.
<code>mphi_<ed>_<n>.png</code>	Momento contra curvatura, a varios niveles de compresión. Muestra por qué la compresión cambia la capacidad a momento .

Las fibras contra el cálculo a mano

```
python semana03\verificar_rc.py lt2 1      # una columna
python semana03\verificar_rc.py lt2 9      # un muro
python semana03\verificar_rc.py lt2       # los dos, resumido
```

Calcula los puntos característicos con las fórmulas del curso (bloque de Whitney, β_1 , punto balanceado) y los compara con lo que da el modelo de fibras. **No se espera que den lo mismo:** se espera que cada diferencia se pueda explicar, y el script la explica.

4.6 El laboratorio de la Semana 3

```
python semana03\lab_semana03.py           # edificio de Ingeniería
python semana03\lab_semana03.py lt2
python semana03\lab_semana03.py conjunto
```

Cubre las Partes A (carga viva), B (sismo) y C (superposición). **Los parámetros que dicte el profesor se pasan por línea de comandos**, sin editar ningún archivo:

Opción	Qué cambia	Ejemplo
--uso	Una fila de la Tabla 4 de NCh1537	--uso oficinas
--q	Sobrecarga de uso en kN/m ² , un número cualquiera	--q 2.5
--cs	Coefficiente sísmico basal	--cs 0.20
--fq	Fracción de Q en el peso sísmico	--fq 0.25
--patron	potencia , nch433 o manual	--patron nch433
--k	Exponente de potencia : 0 uniforme, 1 triangular, 2 el tope de ASCE 7	--k 2
--fracciones	Reparto manual, de abajo hacia arriba	--fracciones 5 10 20 30 35
--comb	Los cuatro factores $\lambda_G \lambda_Q \lambda_{EX} \lambda_{EY}$	--comb 1.2 1.6 1.0 0.3
--combinacion	Una de las declaradas, por nombre	--combinacion 1.2G+1.6Q

```
python semana03\lab_semana03.py ingenieria --cs 0.20 --k 2
python semana03\lab_semana03.py ingenieria --patron nch433
python semana03\lab_semana03.py lt2 --patron manual --fracciones 5 10 20 30 35
python semana03\parametros.py --uso pasillos --comb 1.2 1.6 0 0 # solo muestra qué quedaría
```

De dónde sale cada valor por defecto

La **carga viva** por defecto es **3.0 kN/m²**: NCh1537 Of.2009, Tabla 4, salas de clases — el uso predominante de una facultad. `--uso` cambia de fila (pasillos 4.0, oficinas 2.5, uso público 5.0, techo de mantención 1.0) y `--q` pone cualquier número, que queda marcado como **dictado**. Si el archivo de parámetros declara un uso y un `q` que no calzan con la tabla, el programa se detiene: un número sin fuente no pasa como si fuera de norma.

Los dos repartos en altura no son lo mismo, y conviene no confundirlos. `potencia` es la forma de **ASCE 7 12.8.3** ($F_i \propto w_i \cdot h_i^k$, con k entre 1 y 2 según el período: por eso k = 2 es el tope de ASCE, **no** de NCh433). `nch433` es el artículo **6.2.6** de la norma chilena, que no usa exponente: $A_k = \sqrt{1 - Z_{(k-1)}/H} - \sqrt{1 - Z_k/H}$.

4.7 Regenerar el modelo desde cero

Solo hace falta si se tocaron los planos, el perfil o el código del modelo. **El orden importa:** el conjunto se arma después de los dos cuerpos.

```
python edificios\lt2\armar.py                # 1. geometría → modelo
python comun\calcular.py lt2                 # 2. modelo → resultados
python edificios\lt2\exportar_unity.py        # 3. → data/unity/lt2.json

python edificios\ingenieria\armar.py          # lo mismo, otro cuerpo
python comun\calcular.py ingenieria
python edificios\ingenieria\export_unity.py

python edificios\conjunto\armar.py            # 4. une los dos
python comun\calcular.py conjunto
python edificios\conjunto\exportar_unity.py
```

Volver a leer los planos DXF (lento, y necesita la carpeta `Planos/`):

```
python edificios\lt2\planos\extraer.py "..\Planos\LT2_CAL_dxf" ^
--perfil lt2_2024_22 --salida data\geometria\lt2.json
```

4.8 Verificaciones sueltas

```
python comun\sismo.py lt2                  # los dos casos de sismo
python comun\sismo.py lt2 EY --detalle      # cuánto del borde es giro
python comun\combinar.py lt2                # las 5 combinaciones
python comun\verificar_tributarias.py        # la losa, los 3 edificios
python edificios\lt2\verificar_lt2.py        # las 59 del LT2
python edificios\conjunto\verificar_conjunto.py
python comun\test_contrato_unity.py lt2      # el contrato con Unity
python semana03\verificar_viga_partida.py ingenieria 373
```

5. Lo que hay que saber explicar

Cinco convenciones que no se rompen

1. **Z es vertical en OpenSees, Y en Unity.** La conversión es `Unity(x, z, y)`. Si el edificio se ve acostado, es esto.
2. **Iz es la inercia de gravedad** (la del canto). El motor cruza `Iy` e `Iz` solo en las barras que *no* son verticales.
3. **Las fuerzas internas se leen con** `eleResponse(tag, 'localForce')`, nunca con `eleForce`, que devuelve ejes globales. Esto engañó al proyecto durante toda la Semana 1.
4. **Un nodo de diafragma reacciona también a su restricción**, que es interna. Se separa por grado de libertad o el corte basal sale al doble.
5. **Lo supuesto se declara en** `perfiles/*.json`, con su razón escrita, nunca escondido en el código.

Números de control

Qué	Valor
LT2	232 nodos, 378 elementos, G = 34 148.98 kN
Ingeniería	326 nodos, 559 elementos, G = 50 652.2 kN
Conjunto	558 / 937, G = 84 801.2 kN — la suma exacta ; junta 0.050 m
Torsión del LT2 bajo EY	cociente 1.70 → torsión extrema (NCh433 marca 1.4); excentricidad 35 % del ancho
Columna P.70x70	Mn(P = 0) = 988 kN·m; tracción pura: fibras = cálculo a mano, exacto
Benchmark de Semana 1	UZ techo = -0.0635 mm (SAP2000 dio -0.06375: 0.4 % de diferencia)

Si algo se ve raro en el visor

Se ve	Es
Una viga hundida en el medio	La escala gráfica ×300. 6.55 mm sobre 10 m se dibujan como 1.96 m. Bajar la escala con el control del panel. Comprobado con <code>verificar_viga_partida.py</code> .
El edificio acostado	El cambio de ejes Z↔Y.
La deformada plana	Un campo del C# que no calza con el JSON: <code>python comun\test_contrato_unity.py <ed></code>
Datos viejos	No se regeneró. <code>lanzar_unity.py app <ed></code> lo hace todo.
Huecos blancos donde debería haber losa	<code>python comun\verificar_tributarias.py</code>

Reparto del grupo

Pedro — el LT2 y `comun/` · **Eduardo** — el edificio de Ingeniería y el visor · **Monse** — Semana 3.

Una carpeta tiene un dueño. `comun/` y `unity/` son compartidos: se avisa antes de tocarlos. Y antes de subir cualquier cambio, `python comun\verificar_todo.py`.